

QUARTERLY AWARDS · INNOVATION

Enterprise Risk Management (ERM)

AI-assisted migration from Slate to Airbus DevX (React + OSDK)

[Team name] · [Presenter names] · [Date]

What ERM does and how it works

ERM helps stakeholders understand risks, decision bases and required actions. The ERM dashboard builds ERM reports from Active Risk Manager (ARM) data in Palantir Foundry Skywise.

Single place for ARM data

Live view of all ARM risks and opportunities, with role-based access

Custom dashboards

Group the risks and opportunities you need into dedicated dashboards

Live editing and trend following

Risk managers update key parameters and follow status over time

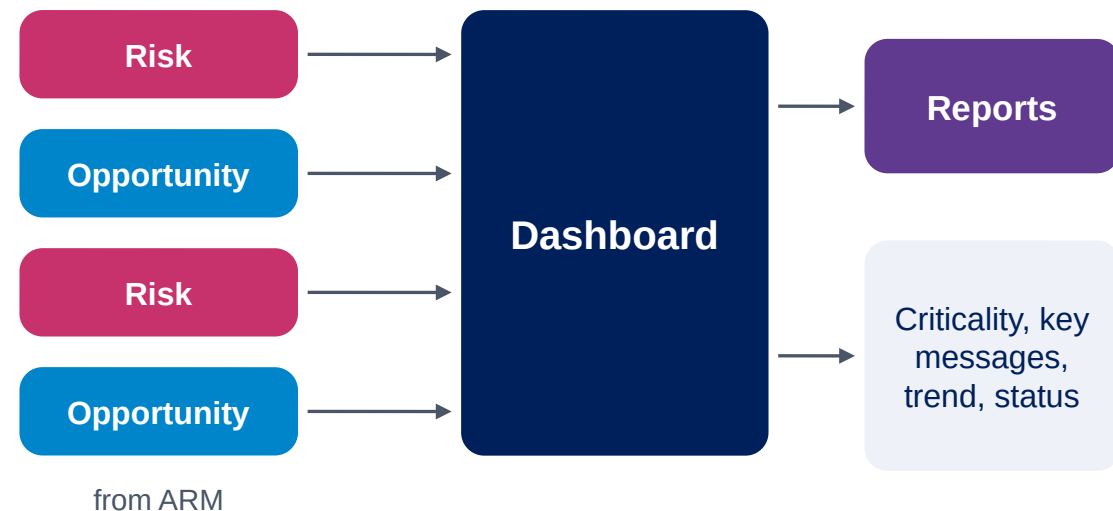
Structured release workflow

Formal, governed quarterly review and release

Cross-team collaboration

Share and co-manage individual risks across divisions

HOW DATA BECOMES A REPORT



DATA PATH



Hundreds of risk managers rely on ERM

892

lifetime unique users across engineering, programmes and operations

320+

monthly active risk managers at peak

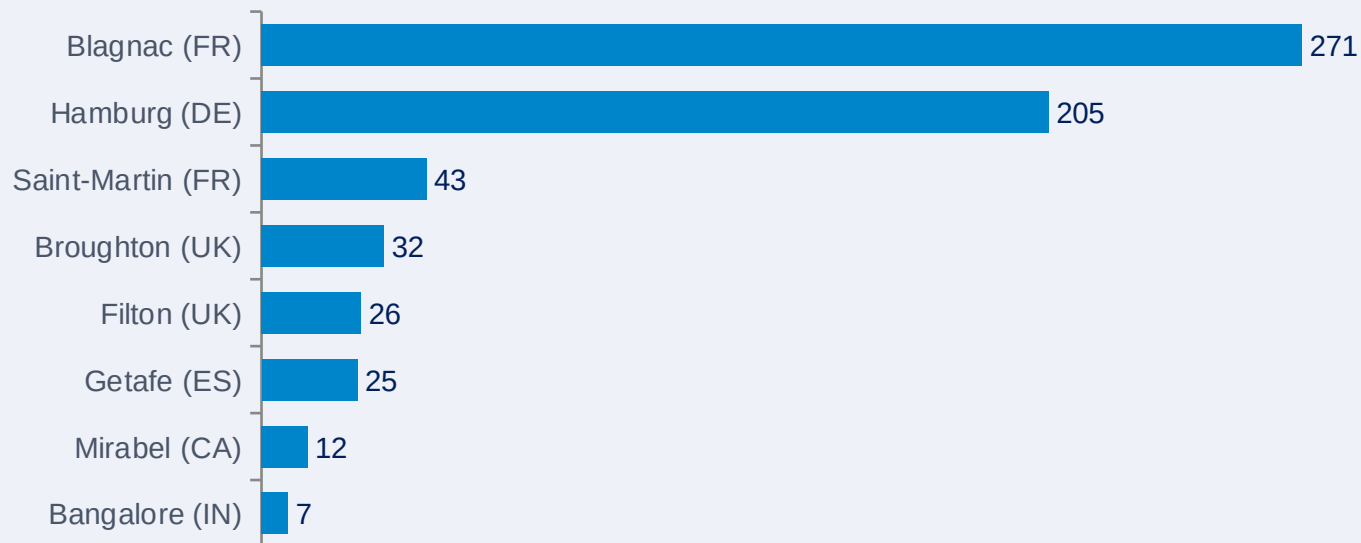
1,600+

custom dashboards created to date

10+

dashboards updated every day by risk owners

ERM users by site



Why it had to be done right

ERM is part of how Airbus manages risk. A migration that broke a feature would disrupt reporting for hundreds of risk managers across sites.

Why migrate: Slate had become the bottleneck

Tangled dependencies

One user action cascades through widgets, events, actions and variables

High regression risk

Changing one feature can break unrelated dashboard behaviour

Slow navigation

Variable trees are recalculated on every tab switch

No documentation

8 Slate exports, about 25 MB of configuration, no specs

One big undocumented app

All behaviour hidden in Slate configuration

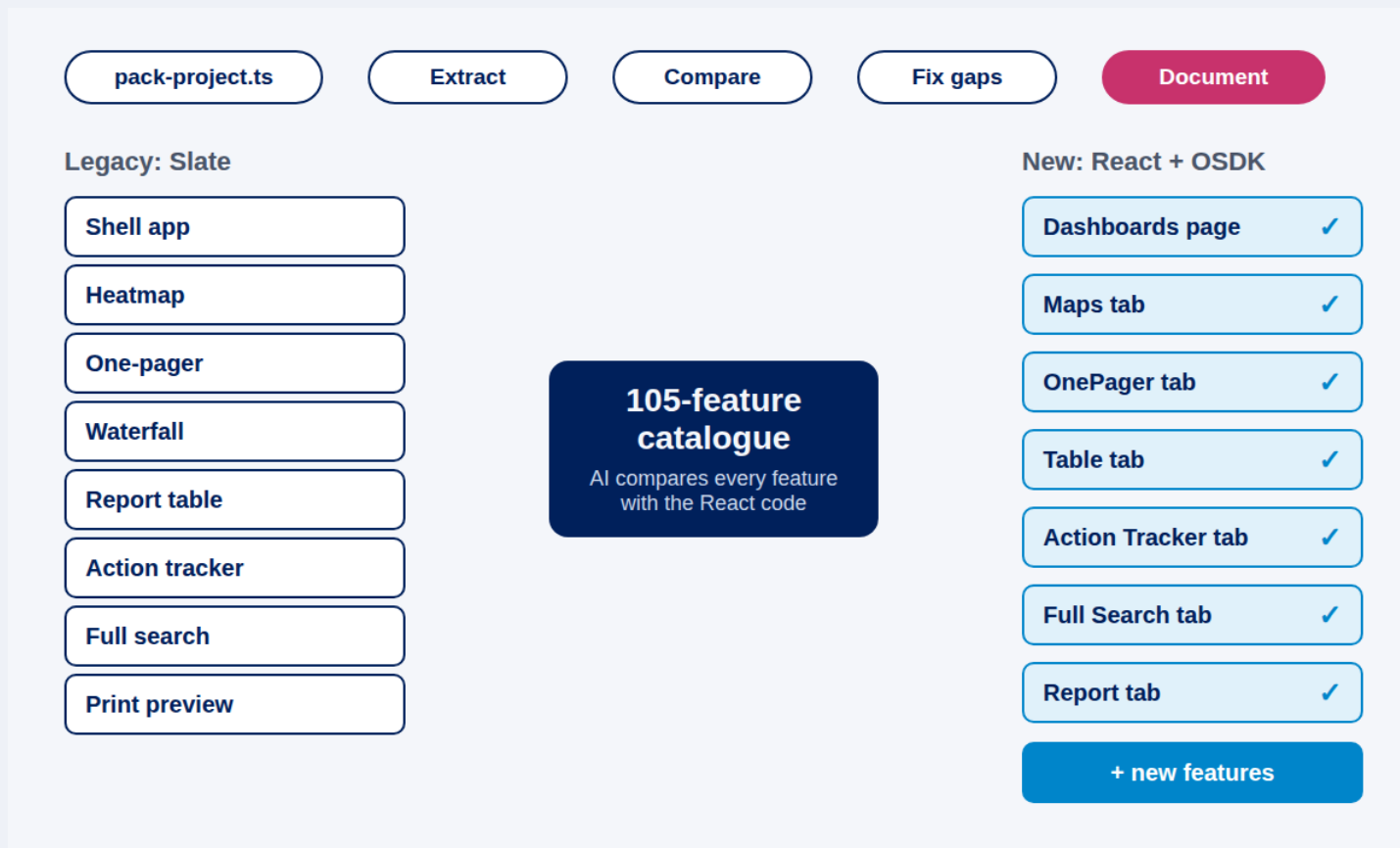
ERM Slate app

8 JSON exports · about 25 MB · no documentation

The Airbus DevX solution: React + OSDK

Clean, modular TypeScript components: isolated modules, safe updates, instant navigation

Our method: AI drafts, people check



pack-project.ts

Our script packs the whole React app, 75 files, into one dated file the AI reads in one go

1. Pack the context
2. Extract legacy features
3. Compare with the React code
4. Fix gaps, re-pack, re-check
5. Document the new app

The process in numbers

1 PI

Whole migration done in one SAFe Program Increment

105

legacy features recovered, with no docs

8

Slate exports analysed, about 25 MB

75 → 1

React files packed into one AI context

31

gaps found and prioritised by AI review

7

security and data issues caught in review

5

enhancements beyond the old app

0 → 10

documents: 8 diagrams and 2 guides

Effort saved: about 76% versus manual

~17

Manual estimate

sprints, $\approx$ 340 person-days

4

Actual with AI

sprints, in one PI

~76%

Effort saved

$(17 - 4) \div 17$ sprints

Where the AI saved the most time

Discovery

105 features catalogued from the exports, with no docs to read

Gap analysis

Every feature checked against the packed React code

Documentation

8 architecture and lifecycle diagrams produced alongside the work

Generic estimate, not a measured baseline: 105 features $\times$ (0.5 day discovery + 2 days build + 0.5 day parity testing) + 25 days documentation $\approx$ 340 person-days. Team of 2 developers, 2-week sprints = 20 person-days per sprint; 1 PI = 4 development sprints.

Our contribution to the 2026 objectives

OUR PROJECT

EVIDENCE

2026 OBJECTIVE

AI-assisted migration of ERM

Slate to React + OSDK
2 developers · 1 PI
pack-project.ts

AI read 8 Slate exports and the packed React code: 105 features recovered, 31 gaps found

Delivered in 1 PI by 2 developers, about 76% less effort than a manual estimate

All 105 features tracked for parity; ERM moved off legacy Slate onto a standard, maintainable stack

pack-project.ts and the playbook let any team repeat the method

Leverage AI to solve business problems

India TFO · Digital: democratise AI

Deliver on time, on cost, on quality

Digital: optimise our digital resources

Ensure our operational performance

Digital: availability of systems for operations

Grow digital acumen and competencies

Digital: develop our people · India TFO

A playbook any team can reuse

Built together

- Business users confirmed the feature catalogue
- Data and security owners kept restricted views intact
- Backend and frontend fixes shipped together

Ready to transfer

- Other Slate apps in [business areas]
- Any low-code tool with little documentation
- Reusable as is: pack-project.ts, comparison template, diagrams

Pack. Extract. Compare. Fix. Document. Repeat for the next app.